A function that displays the summary statistics (mean, N, and std. dev.) for 2 samples.

```python
import math

prozac = [64, 68, 84, 65, 50, 60, 81, 76]
truzac = [53, 44, 61, 50, 40, 39, 58, 42]

def summary_stats(data1,data2 , name1="Sample 1", name2="Sample 2"):
    def mean(data):
        return sum(data) / len(data)

    def std_dev(data):
        m = mean(data)
        return math.sqrt(sum((x - m)**2 for x in data) / (len(data) - 1))

    print("---- Summary Statistics ----")
    print(f"{name1}:")
    print("N =", len(prozac))
    print("Mean =", mean(prozac))
    print("Std Dev =", std_dev(prozac))
    print()

    print(f"{name2}:")
    print("N =", len(truzac))
    print("Mean =", mean(truzac))
    print("Std Dev =", std_dev(truzac))
    print()

summary_stats(prozac,truzac, "Prozac", "Truzac")
```

After running this python code we get the following statistics:
---- Summary Statistics ----
Prozac:
N = 8
Mean = 68.5
Std Dev = 11.338934190276817

Truzac:
N = 8
Mean = 48.375
Std Dev = 8.399617338222702

This is the same as the answer in Coolidge which gave N = 8 for both and Mean = 68.50 for Prozac and Mean = 48.375 for Truzac.
Another function that displays statistics for an independent Sample T-test.

```
def independent_ttest(data1, data2):
    t_stat,p_value = ttest_ind(data1, data2, equal_var=True)
    df = len(data1) + len(data2) - 2
    print("t =", t_stat)
    print("df =", df)
    print("p =", p_value)
    if p_value < 0.05:
        print("reject")
    else:
        print("fail to reject")

independent_ttest(prozac, truzac)
```

I used from scipy.stats import ttest_ind and ttest_ind. for this part. These are the statistics I got when I ran my function:
t = 4.033833683615182
df = 14
p = 0.0012316218035785958
reject

These are the same as the Coolige.

2. Histograms plots of Horseshoe Crab **width** with satellite and without satellite

Code Snippet:

```
width_with = []
width_without = []

weight_with = []
weight_without = []

with open("HorseshoeCrabDataMod.csv") as file:
    crabreader = csv.DictReader(file)
    for row in crabreader:
        width = float (row ["width"])
        weight = float (row ["weight"])
        sat = int(row["y"])

        if sat > 0:
            width_with.append(width)
            weight_with.append(weight)

        else:
           width_without.append(width)
           weight_without.append(weight)
```

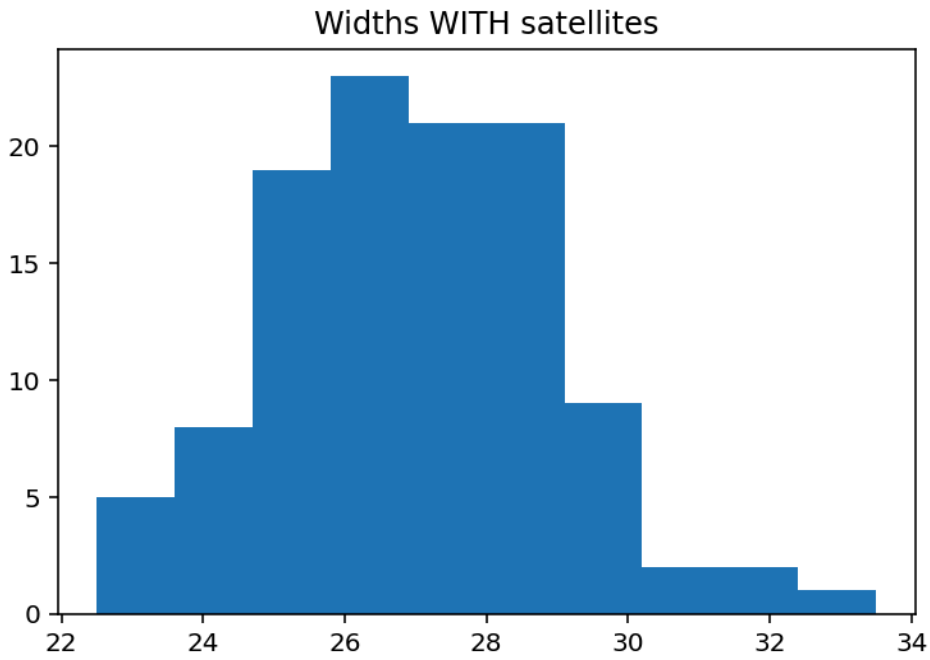
```
# Widths
pylab.hist(width_with)
pylab.title("Widths WITH satellites")
pylab.show()

pylab.hist(width_without)
pylab.title("Widths WITHOUT satellites")
pylab.show()

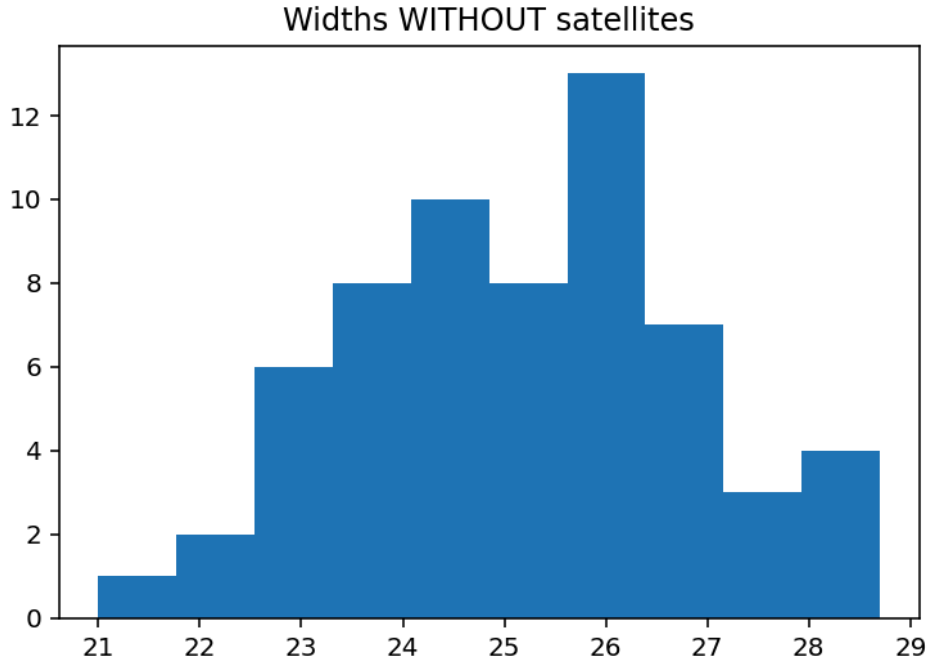
# Weights
pylab.hist(weight_with)
pylab.title("Weights WITH satellites")
pylab.show()

pylab.hist(weight_without)
pylab.title("Weights WITHOUT satellites")
pylab.show()
```

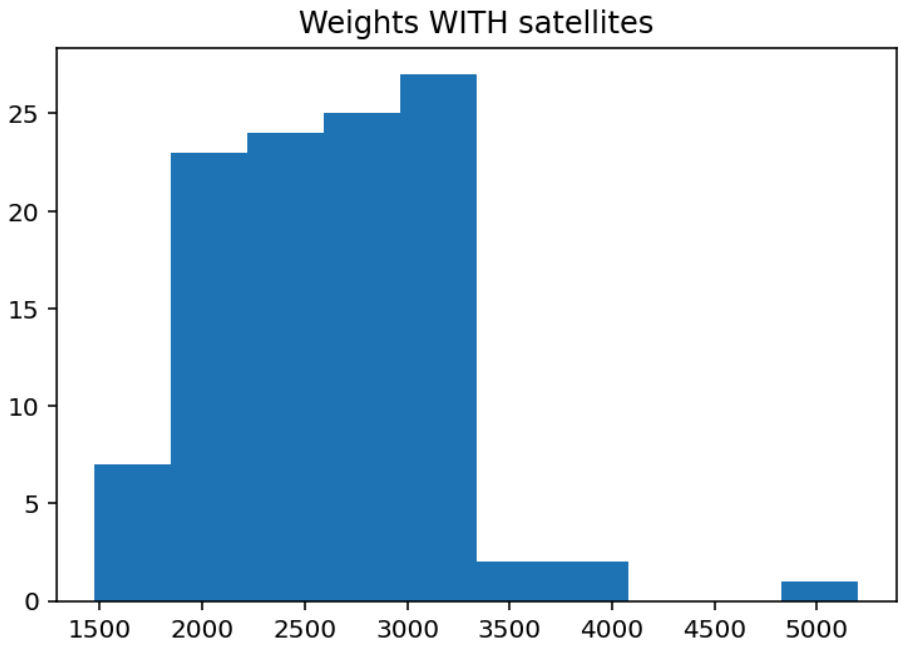
With satellite:



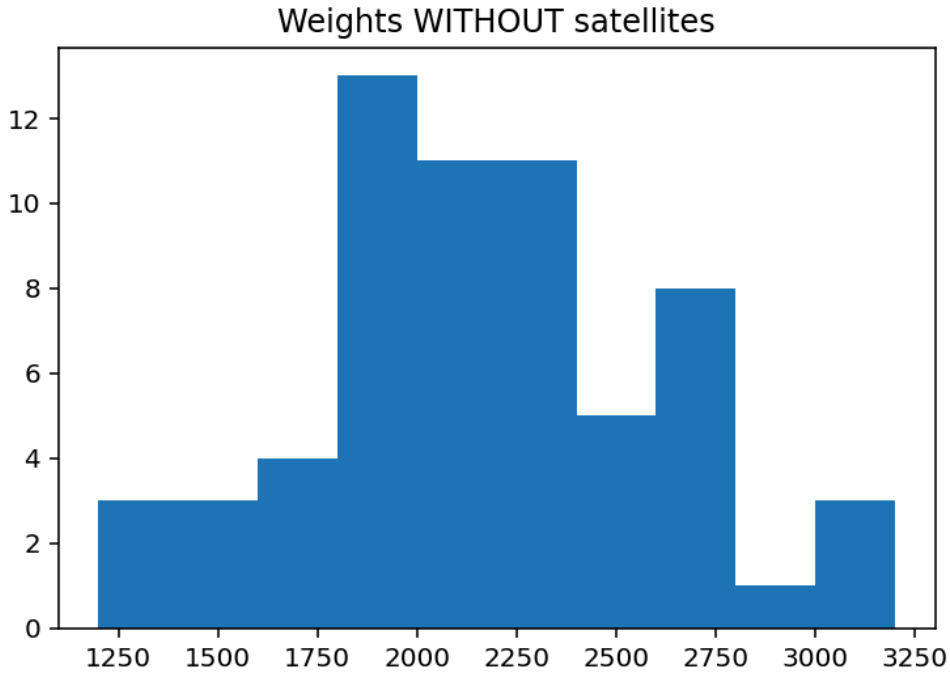
Without satellite:



3. Histograms plots of Horseshoe Crab **weights** with satellite and without satellite
With satellite:



Without satellite:



4. Random 20 samples from the **widths** for each dataset (satellite and without satellite) in separate lists

Width (**with**):
N = 8
Mean = 68.5
Std Dev = 11.338934190276817

Width (**without**):
N = 8
Mean = 48.375
Std Dev = 8.399617338222702

t = 3.4864968274694164
df = 38
p = 0.0012519396218797403
Reject

5. Random 20 samples from the **weights** for each dataset (satellite and without satellite) in separate lists

Weight (**with**):
N = 8
Mean = 68.5
Std Dev = 11.338934190276817

Weight (**without**):
N = 8
Mean = 48.375
Std Dev = 8.399617338222702

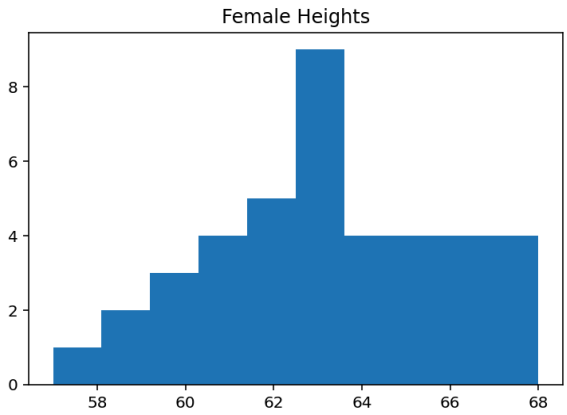
t = 4.132972606917056
df = 38
p = 0.0001899494976517995
Reject

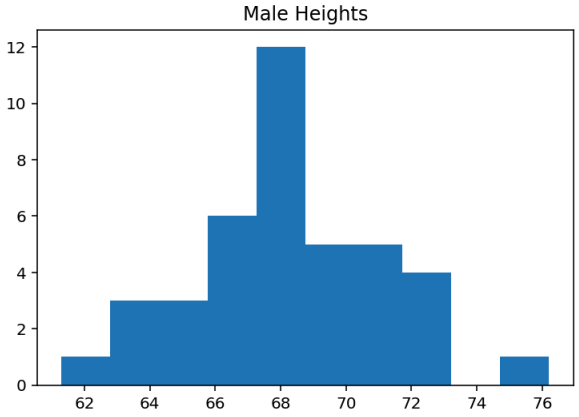
Code Snippet:

```
sample_width_sat = random.sample(width_with, 20)
sample_width_no_sat = random.sample(width_without, 20)
summary_stats(sample_width_sat, sample_width_no_sat,
              "Width (Sat)", "Width (No Sat)")
independent_ttest(sample_width_sat, sample_width_no_sat)

sample_weight_sat = random.sample(weight_with, 20)
sample_weight_no_sat = random.sample(weight_without, 20)
summary_stats(sample_weight_sat, sample_weight_no_sat,
              "Weight (Sat)", "Weight (No Sat)")
independent_ttest(sample_weight_sat, sample_weight_no_sat)
```

6. Analyze your own data: From MathCS.org - Statistics I found two datasets, Female health and Male health.
Null Hypothesis: There isn't a difference between female and male height average
Hypothesis: There is a difference between males and the height of females.
Histograms:





Statistics:

Female:
Total female data: 40
Female sample size: 20
N = 20
Mean = 62.74500000000000.
Std Dev = 2.6154349542666893

Male:
Total male data: 40
Male sample size: 20
N = 20
Mean = 68.42
Std Dev = 2.697093952659731

---

t = -6.755278468918125
df = 38
p = 5.28128932251133
Reject

- The female sample had a mean of 62.75 (N = 20, SD = 2.62), and the male sample had a mean of 68.42 (N = 20, SD = 2.70). I conducted an independent samples t-test. The results showed t(38) = -6.76, p = 5.28. Since the p-value is less than 0.05, we reject the null hypothesis. This means there is a significant difference between male and female heights. So the difference is statistically significant.

```
female_heights = []
male_heights = []

# FEMALE
with open("health_female.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        if row["HT"].strip() != "":
            female_heights.append(float(row["HT"]))

# MALE
with open("health_male.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        if row["HT"].strip() != "":
            male_heights.append(float(row["HT"]))

# CHECK LENGTHS (IMPORTANT)
print("Total female data:", len(female_heights))
print("Total male data:", len(male_heights))
```

```
# HISTOGRAMS
pylab.hist(female_heights)
pylab.title("Female Heights")
pylab.show()

pylab.hist(male_heights)
pylab.title("Male Heights")
pylab.show()

# RANDOM SAMPLE OF 20
female_sample = random.sample(female_heights, 20)
male_sample = random.sample(male_heights, 20)

# CHECK SAMPLE SIZE (VERY IMPORTANT)
print("Female sample size:", len(female_sample))
print("Male sample size:", len(male_sample))

# RUN FUNCTIONS
summary_stats(female_sample, male_sample, "Female", "Male")
independent_ttest(female_sample, male_sample)
```

I also had to update my summary_stats function to:

```
def summary_stats(data1, data2, name1="Sample 1", name2="Sample 2"):
    def mean(data):
        return sum(data) / len(data)

    def std_dev(data):
        m = mean(data)
        return math.sqrt(sum((x - m)**2 for x in data) / (len(data) - 1))

    print(f"{name1}:")
    print("N =", len(data1))
    print("Mean =", mean(data1))
    print("Std Dev =", std_dev(data1))
    print()

    print(f"{name2}:")
    print("N =", len(data2))
    print("Mean =", mean(data2))
    print("Std Dev =", std_dev(data2))
    print()
```